OS UNIX-like

OS UNIX certificato

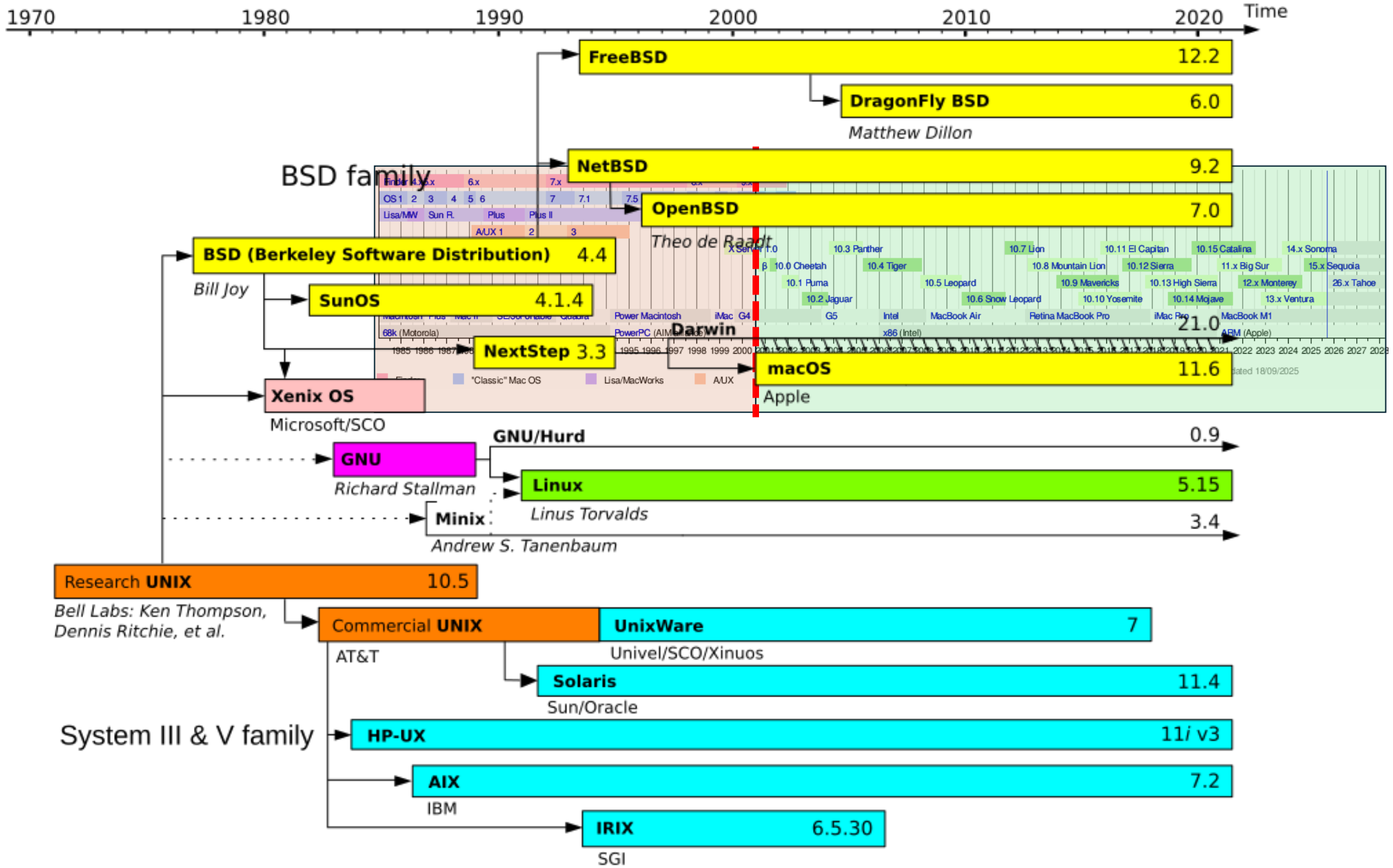
Famiglia: MacOS X / macOS
Versioni: Yosemite, El capitan, Sierra, High Sierra, Monterey, Ventura…

Famiglia: GNU/Linux
Distribuzioni comuni: Debian, Ubuntu, Fedora, Arch…

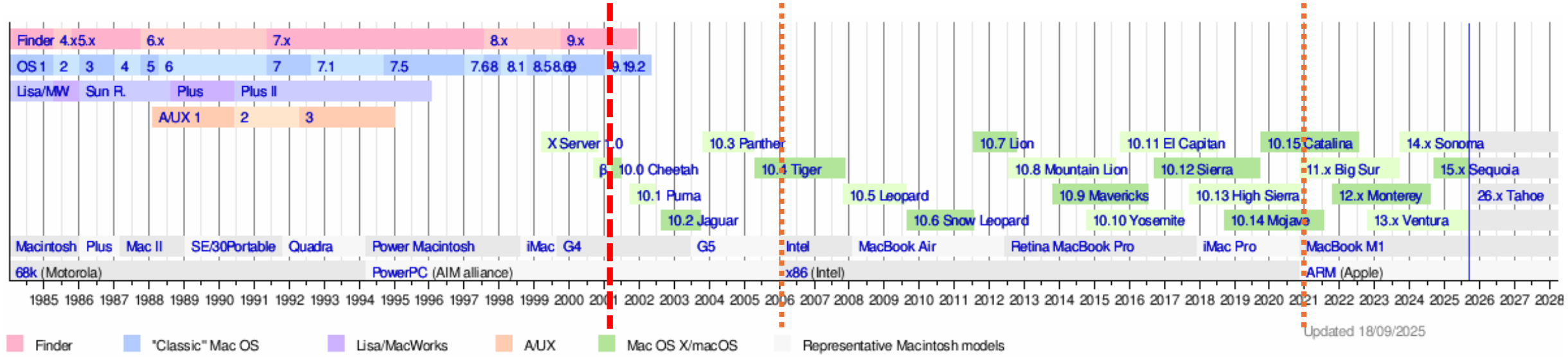
NT

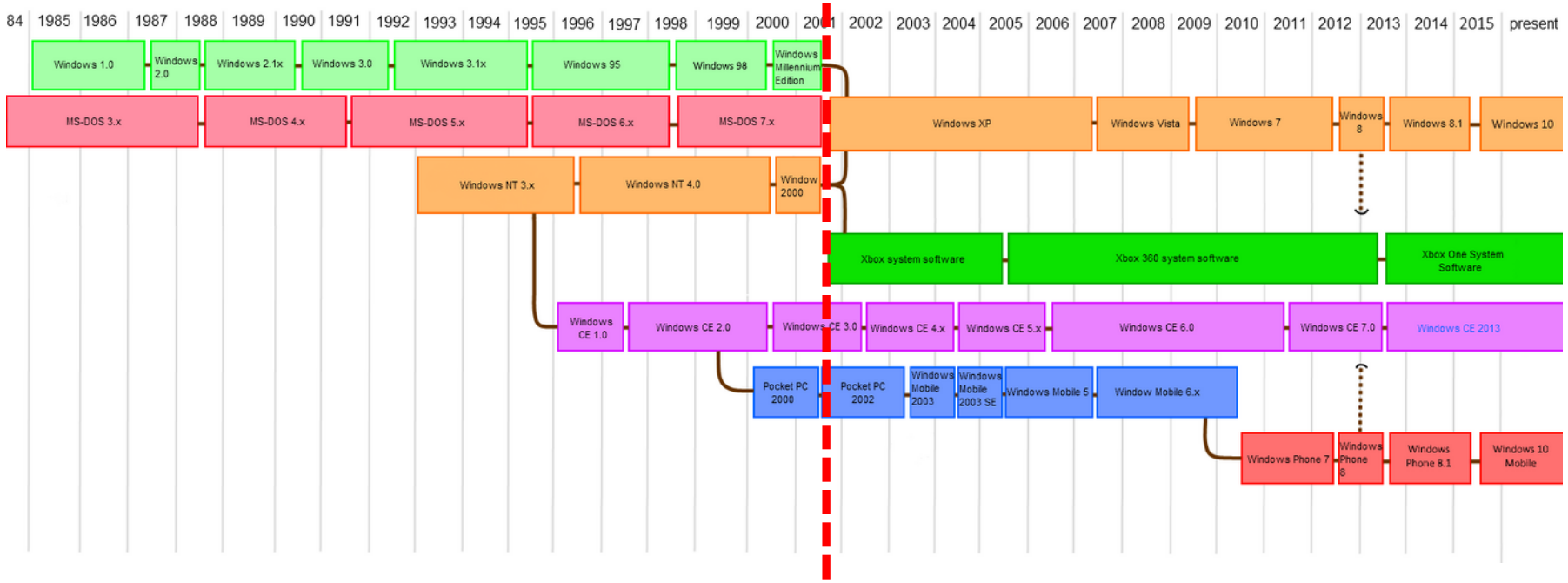
Famiglia: Windows NT
Versioni: Windows 2000, XP, Vista, 7, 8, 8.1, 10, 11…

UNIX/macOS timeline

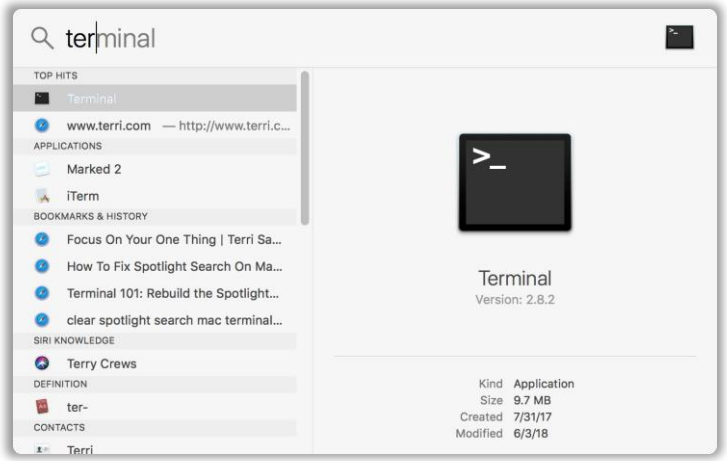


Timeline di macOS e Windows affiancate

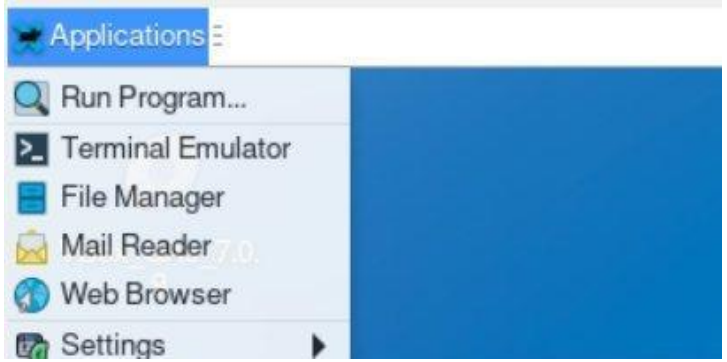


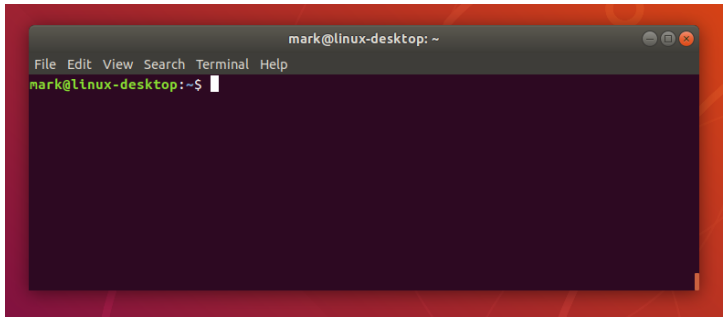


cmd + barra spaziatrice per aprire Spotlight e cercare l’applicazione Terminale



Ctrl + Alt + T è la scorciatoia da tastiera più diffusa su Ubuntu, Debian, Linux Mint e molte altre distro per aprire una nuova finestra del Terminale.
In alternativa è possibile aprire il terminale dal menu della GUI della distro.





Windows non è un OS basato su UNIX e non include la shell bash nativamente o per default (almeno per ora).

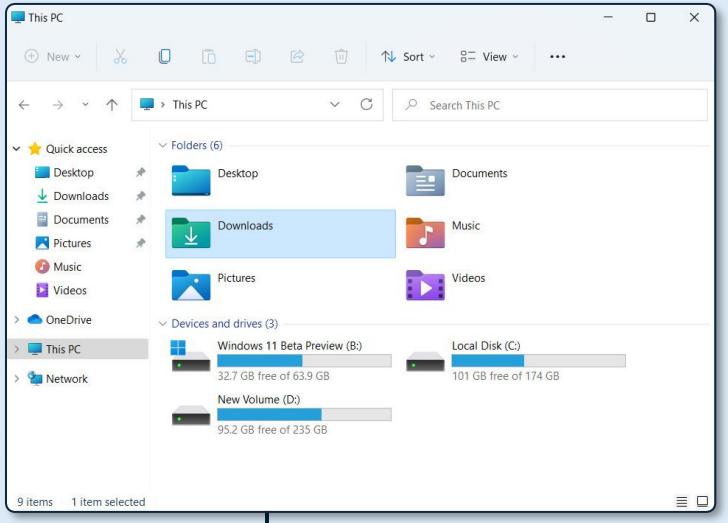
La shell CLI predefinita di Windows è cmd o Windows PowerShell.

Su Windows 10 e 11 l’uso di bash è possibile dopo l’installazione di WSL (Windows Subsystem for Linux)

Per sistemi precedenti si usa una macchina virtuale.

Windows 10 / 11

(NT)



Kernel Linux in VM, File system: ext4.
Diverse distro disponibili. Default: Ubuntu.

/mnt
es.:
/mnt/c/Users/luca11
/mnt/d
/mnt/e

WSL2

/home
es.:
/home/luca
/home/ugo

C: **D:** **E:**

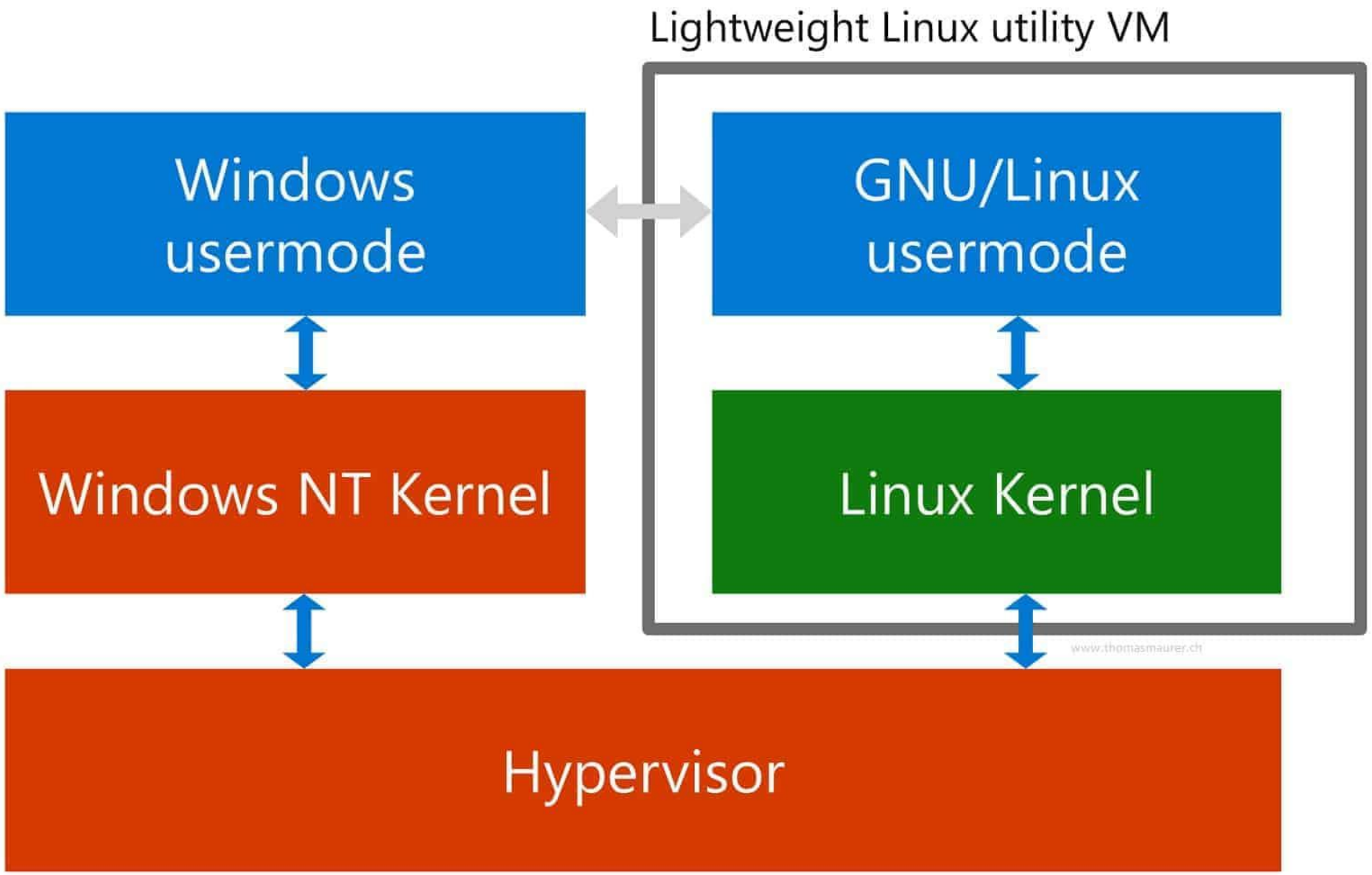
networking

networking

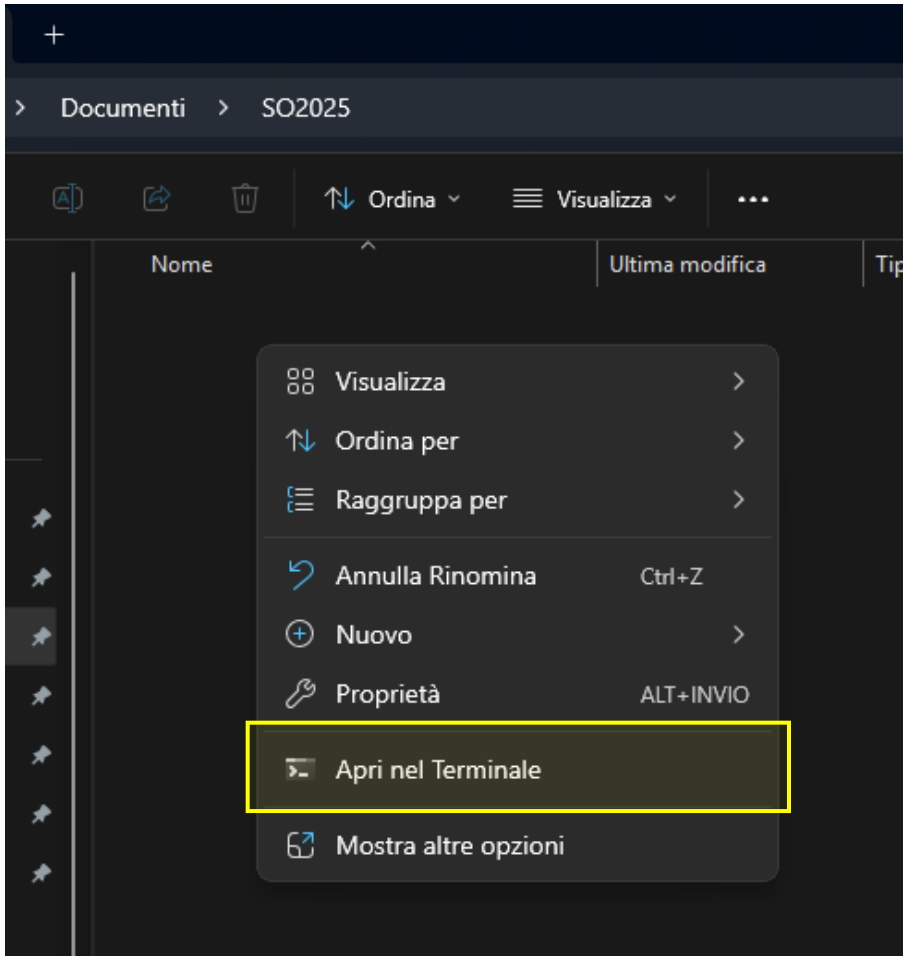
File system: NTFS (sistema/archiviazione).
FAT32, exFAT (solo archiviazione)

Modalità di rete in WSL:
- NAT (predefinita): IP privato, richiede il port forwarding.
- Bridged: stessa LAN dell'host, accesso diretto, richiede configurazione.
- Mirrored (Windows 11+): stesso IP dell'host, la più semplice per lo sviluppo locale.

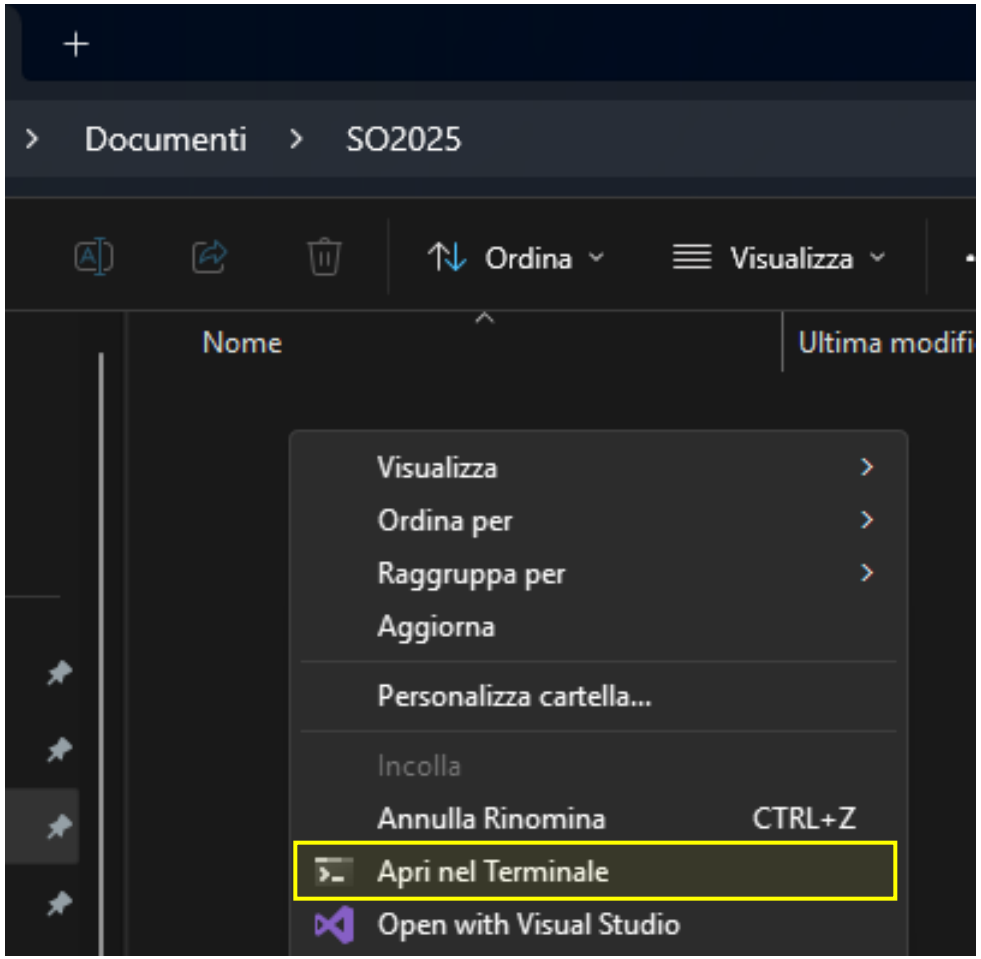
WSL 2 architecture overview



Apertura terminale in una working directory (es. SO2025) su Windows

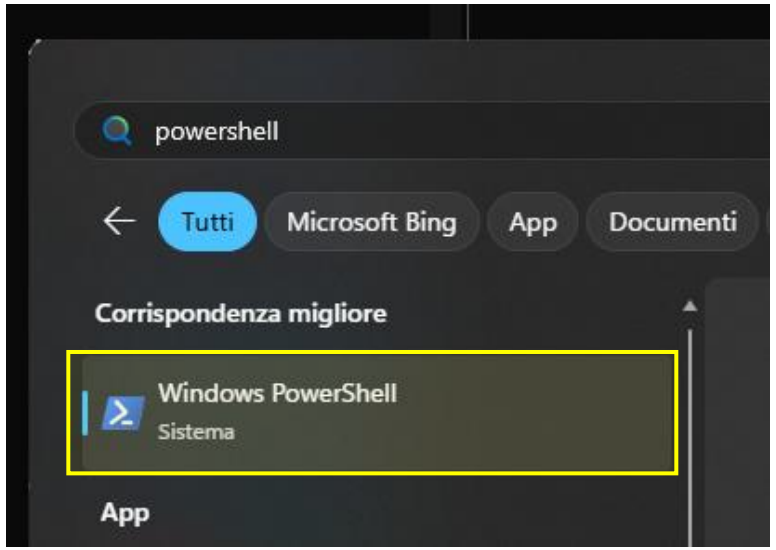


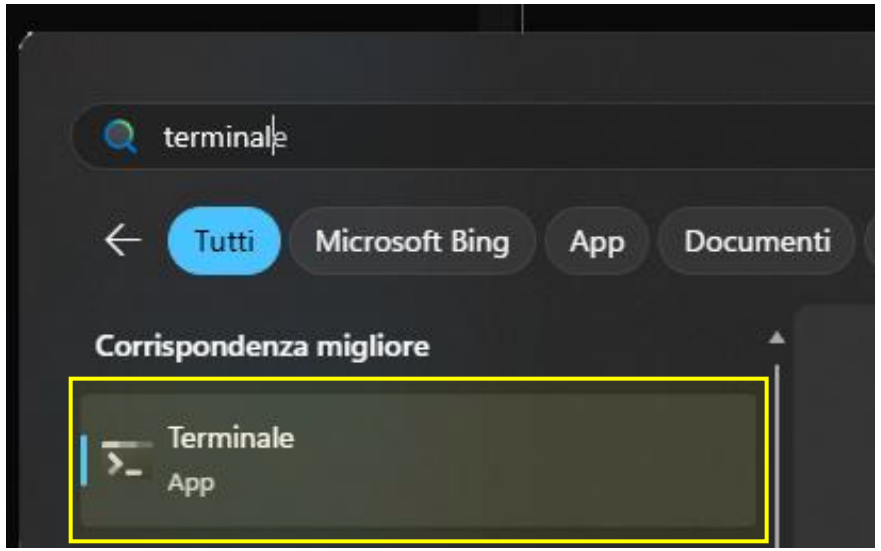
Su Windows 11:
> Tasto destro nella cartella da usare come current working direcrtoty iniziale
> Apri nel Terminale

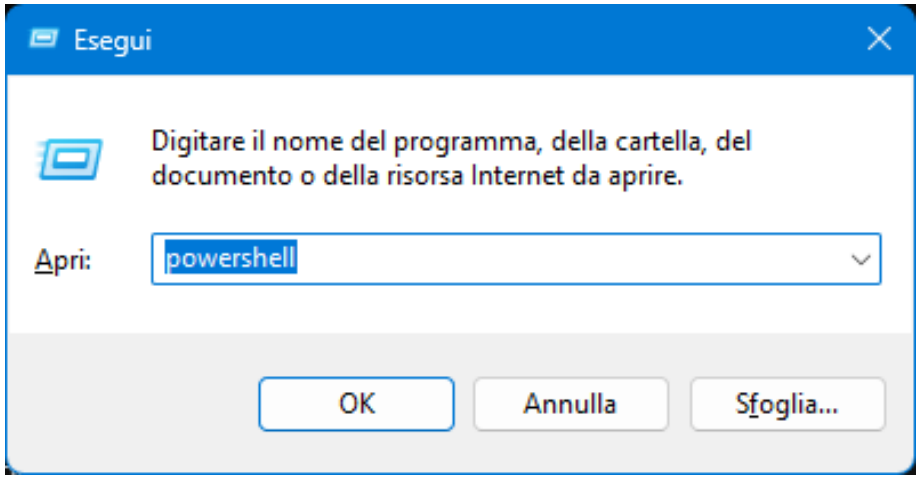


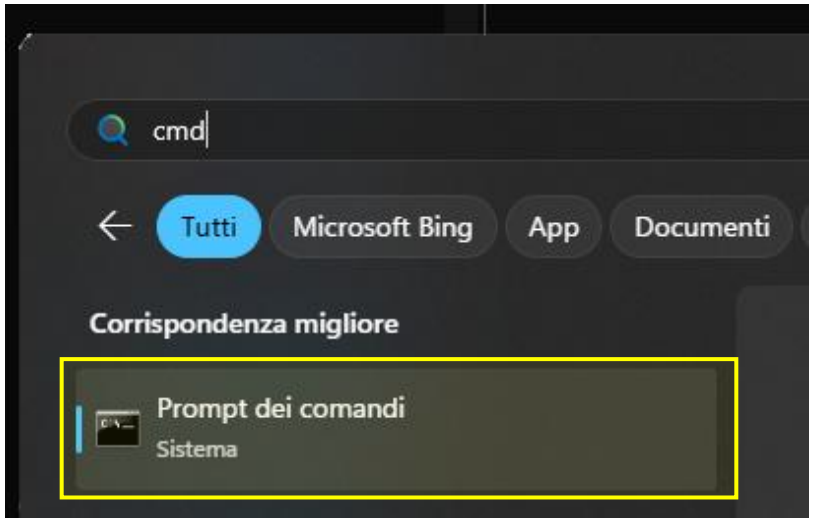
Su Windows 10/11:
> Shift + Tasto destro nella cartella da usare come current working direcrtoty iniziale
> Apri nel Terminale

Alternative apertura terminale su Windows









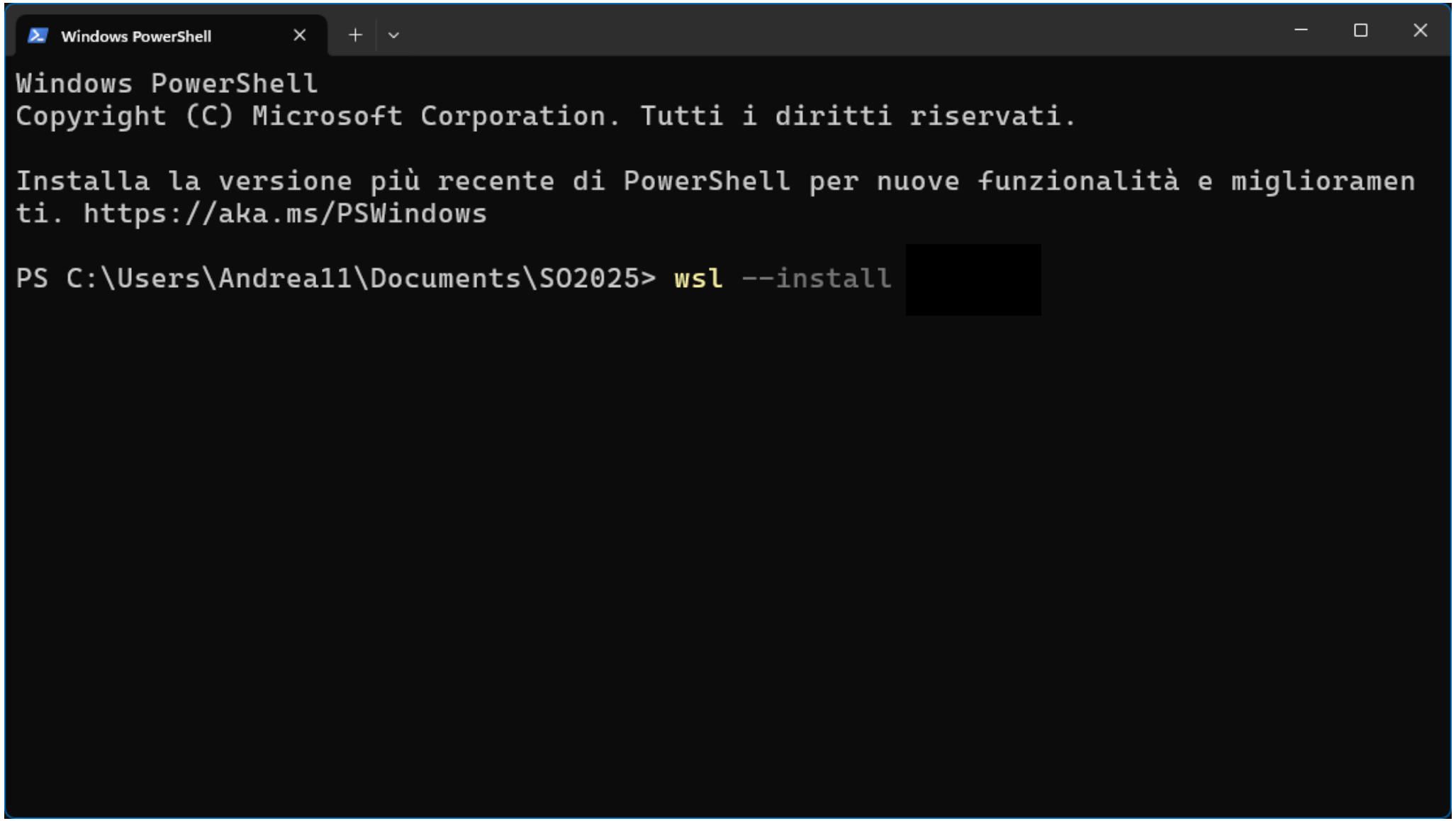
```
achz25@DESKTOP-L8GIS7I: /r

Windows PowerShell
Copyright (C) Microsoft Corporation. Tutti i diritti riservati.

Installa la versione più recente di PowerShell per nuove funzionalità e
miglioramenti. https://aka.ms/PSWindows

PS C:\Users\Andrea11\Documents\SO2025> wsl
achz25@DESKTOP-L8GIS7I:/mnt/c/Users/Andrea11/Documents/SO2025$
```

PowerShell

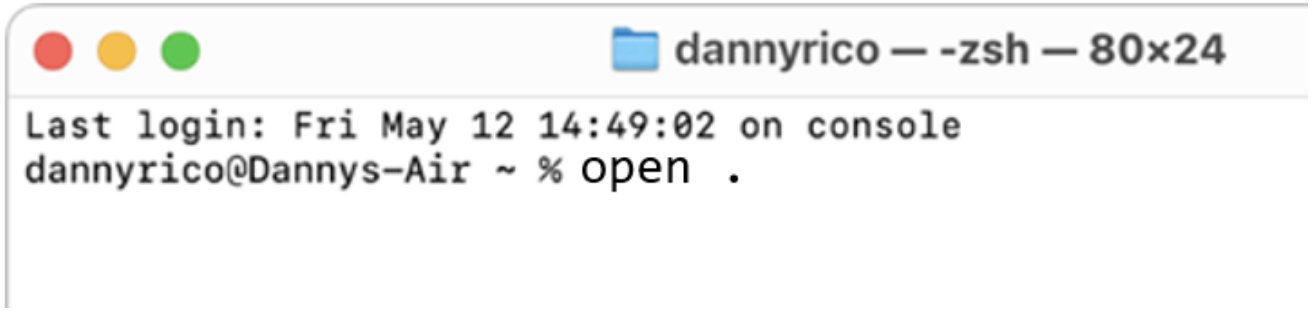
Bash

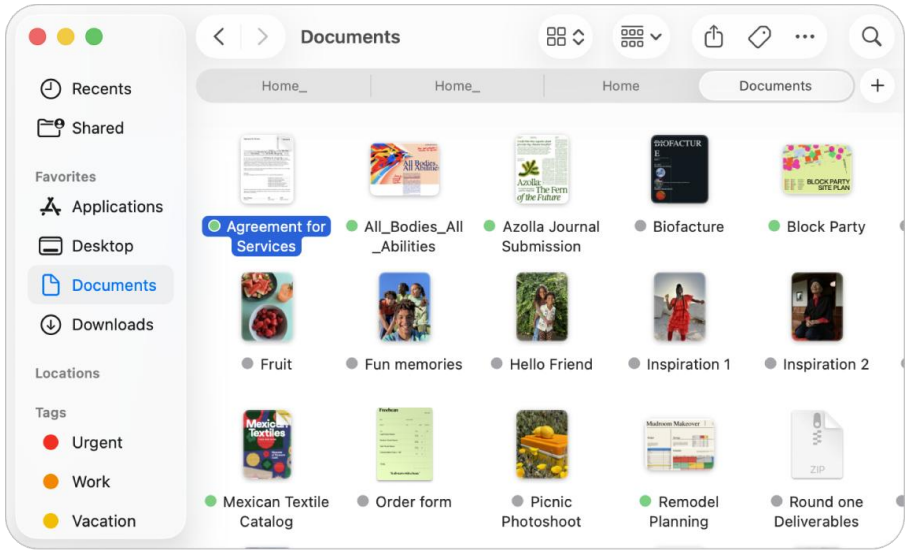
Come installare Linux in Windows con WSL

https://learn.microsoft.com/it-it/windows/wsl/install



Il comando open di macOS su Linux o WSL



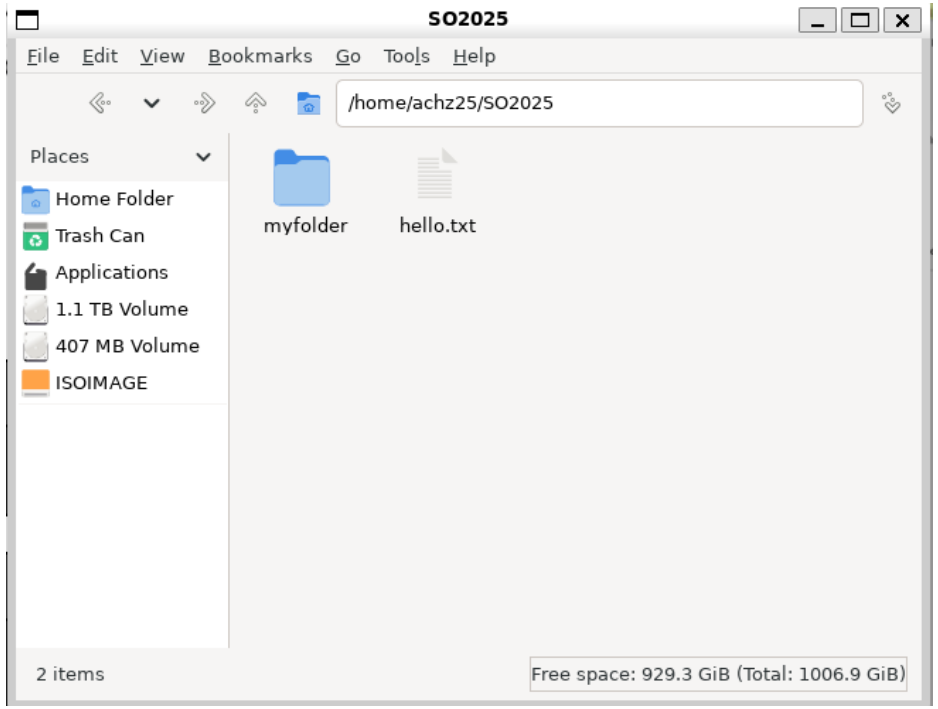


Linux

```
xdg-open . &
```

WSL

```
sudo apt install -y xdg-utils pcmanfm
xdg-open . &
```



Comando con**cat**enate

Visualizzare il contenuto di file di testo sul terminale.
Concatenare più file e mostrarli in sequenza.
Creare nuovi file da input.
Copiare contenuti tra file.

bash

```bash
cat [opzioni] [file ... ]
```

bash

```bash
cat file.txt
```

bash

```bash
cat file1.txt file2.txt ciao.txt
```

Comando con**cat**enate

```bash
cat >> mio_.txt
```

```bash
cat file1.txt > file2.txt
```

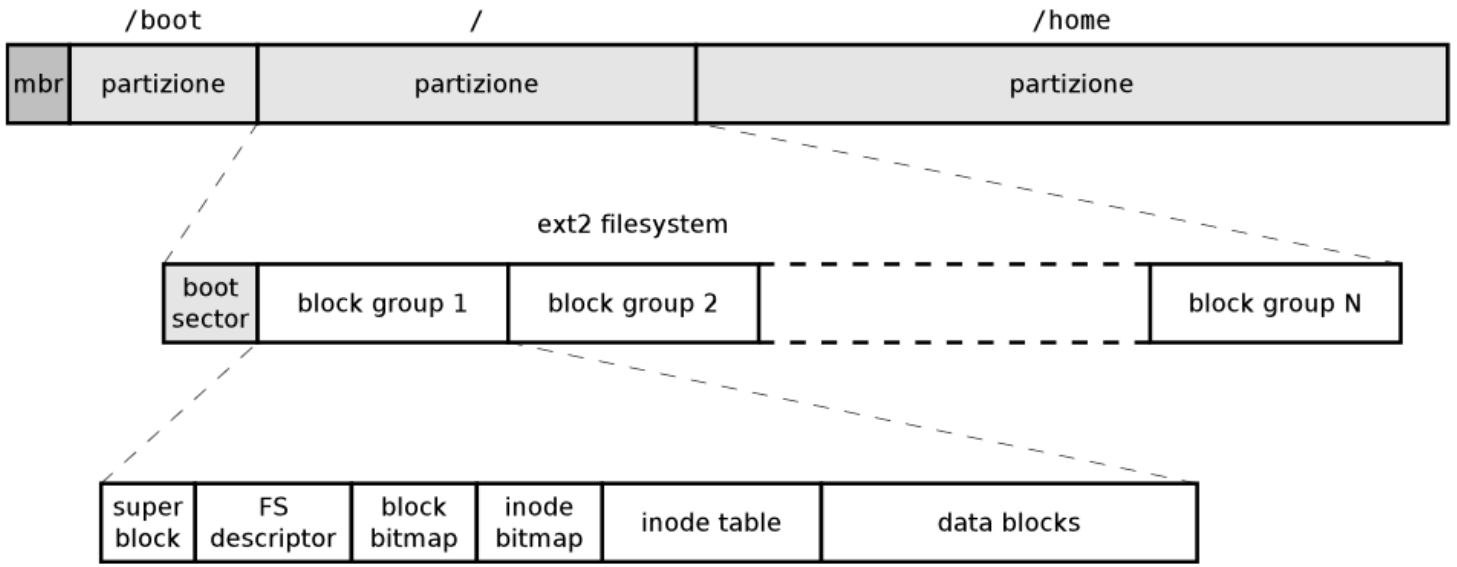
```bash
cat -n file.txt
```

Il filesystem

Un filesystem è una struttura dati che un sistema operativo usa per organizzare e gestire i dati su un dispositivo di memoria (disco rigido, SSD, chiavetta USB, scheda SD, ecc.).
In altre parole, è lo strato intermedio tra i programmi (che vedono file e cartelle) e il disco fisico (che vede solo settori numerati).

Funzioni principali di un filesystem

1. Organizzazione
    1. Trasforma un flusso di byte in file e directory.
    2. Permette una struttura gerarchica (albero di cartelle).
2. Gestione dello spazio
    1. Decide dove scrivere i dati fisicamente.
    2. Tiene traccia di blocchi liberi e occupati (tramite bitmap, tabelle, journaling...).
3. Accesso ai dati
    1. Consente lettura, scrittura, modifica, cancellazione dei file.
    2. Garantisce performance efficienti anche con milioni di file.
4. Protezione e coerenza
    1. Gestisce permessi (chi può leggere/scrivere un file).
    2. In molti filesystem moderni (es. ext4, NTFS, APFS), usa journaling per ridurre i rischi in caso di crash.



Struttura tipica di un Block Group

1. Superblocco di backup

1. Una copia del superblocco principale (non in tutti i gruppi, dipende da come è stato creato il FS).
2. Contiene le info fondamentali del filesystem (dimensione blocchi, inode totali, mount count, ecc.).

2. Group Descriptor Table (parzialmente contenuta all'inizio dei gruppi)

1. Struttura che descrive le info del gruppo stesso.
2. Non sempre duplicata in ogni gruppo.

3. Block Bitmap

1. Una mappa di bit dove 1 = blocco occupato, 0 = blocco libero.
2. Indica quali blocchi del gruppo sono utilizzabili.

4. Inode Bitmap

1. Stesso concetto, ma per gli inode (descrittori dei file).

5. Tabella Inode (Inode Table)

1. Contiene gli inode veri e propri del gruppo.
2. Ogni inode descrive un file/directory (puntatori ai blocchi di dati, permessi, UID/GID, timestamp, ecc.).

6. Blocchi di dati (Data Blocks)

1. La parte "utile" del gruppo: dove vengono salvati i contenuti dei file.
2. Rappresentano la maggior parte dello spazio del gruppo.

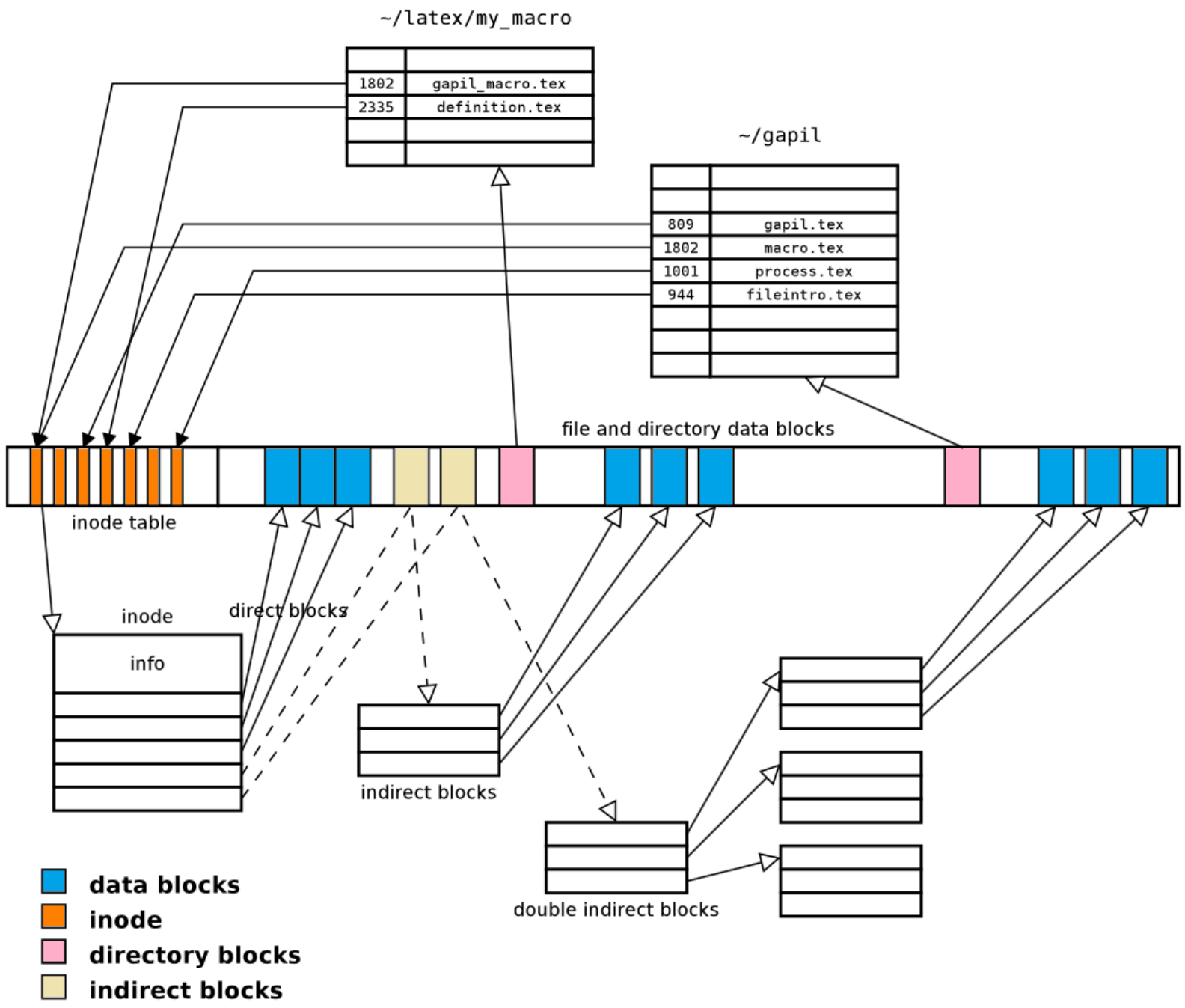
Blocchi di dati (Data blocks)

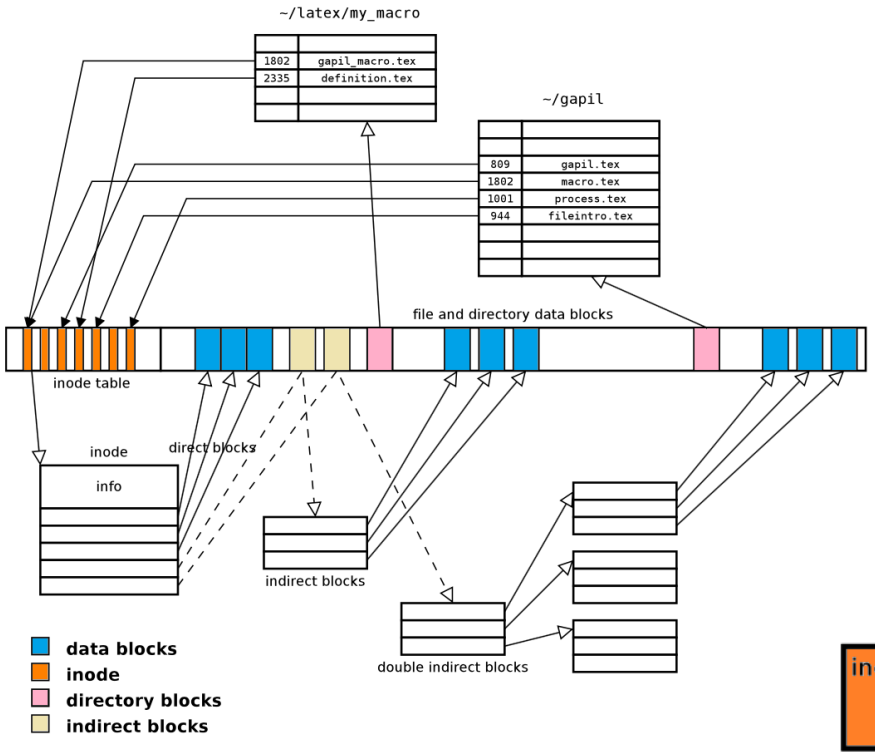
Sono l'unità minima di allocazione dello spazio su disco.

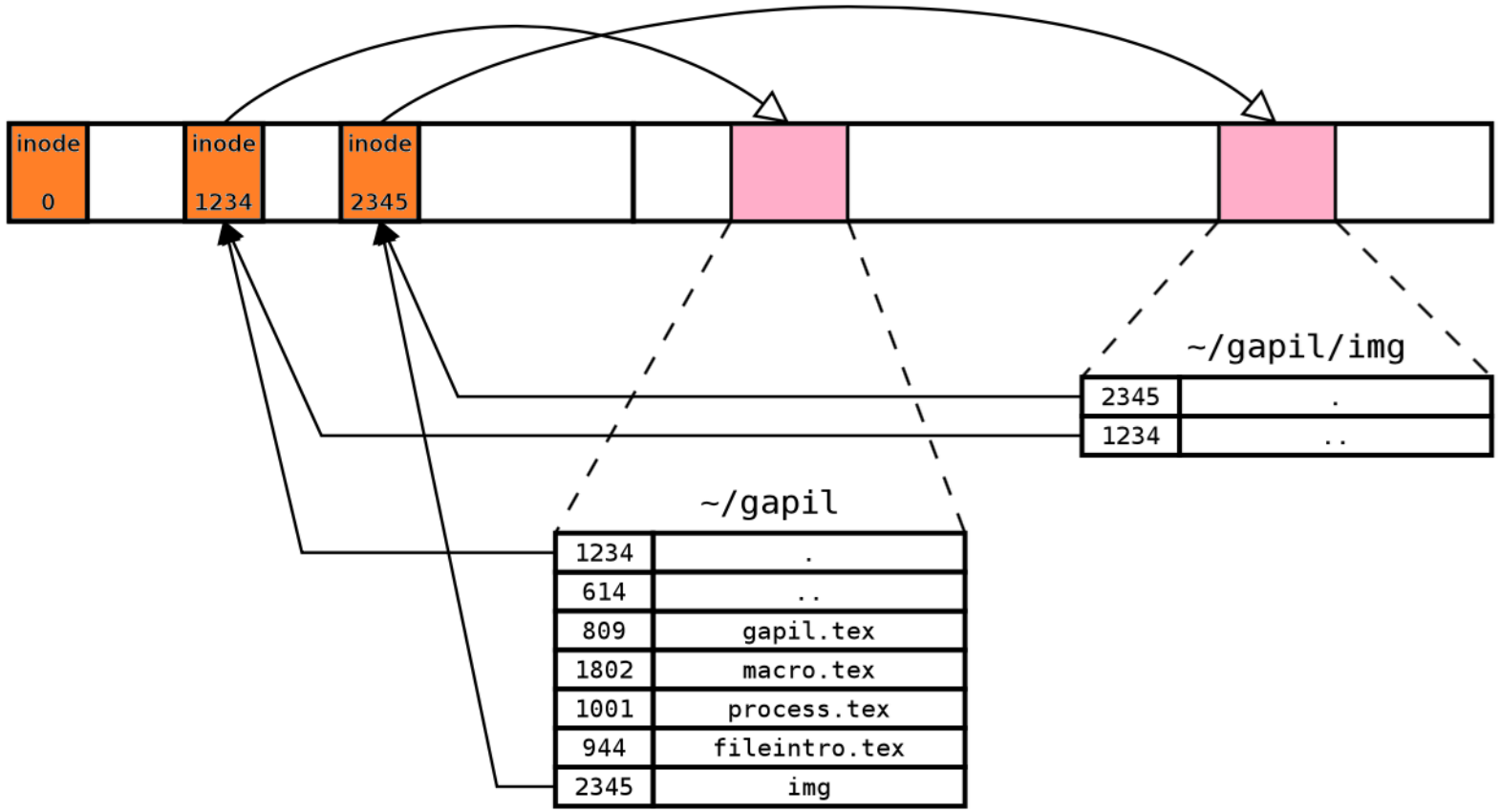
Dimensioni tipiche: 1 KB, 2 KB, 4 KB o 8 KB (il valore più comune nei sistemi moderni è 4 KB).

La dimensione viene scelta al momento della creazione del filesystem (mkfs), in base alla dimensione del disco e all'uso previsto:
Blocchi piccoli: meno spreco di spazio con file piccoli (frammentazione interna minore).
Blocchi grandi: migliori performance con file grandi.

Ogni block group contiene 8x1024 = 8192 blocchi dati.
Dimensione gruppo = 8192 x dimensione singolo blocco dati


~/latex/my_macro
1802
gapil_macro.tex
2335
definition.tex
~/gapil
809
gapil.tex
1802
macro.tex
1001
process.tex
944
fileintro.tex
file and directory data blocks
inode table
inode
info
direct blocks
indirect blocks
double indirect blocks
data blocks
inode
directory blocks
indirect blocks


~/latex/my_macro
1802 gapil_macro.tex
2335 definition.tex
~/gapil
809 gapil.tex
1802 macro.tex
1001 process.tex
944 fileintro.tex
file and directory data blocks
inode table
inode
info
direct blocks
indirect blocks
double indirect blocks
data blocks
inode
directory blocks
indirect blocks


inode 0
inode 1234
inode 2345
~/gapil/img
2345 .
1234 ..
~/gapil
1234 .
614 ..
809 gapil.tex
1802 macro.tex
1001 process.tex
944 fileintro.tex
2345 img

Comando ssh

Il comando ssh (Secure SHell) serve per collegarsi in modo sicuro a un altro computer o server tramite rete, utilizzando la crittografia per proteggere la comunicazione. È uno strumento utilizzato in ambiente Linux, macOS e disponibile anche su Windows (PowerShell/WSL o client SSH).

sintassi

```
ssh [opzioni] utente@host
```

bash

```bash
ssh mario@10.0.14.23
```

Comando scp

scp (Secure Copy), serve per copiare file e directory tra computer locali e remoti usando il protocollo SSH

sintassi

```
scp [opzioni] sorgente destinazione
```

Sorgente: file o directory da copiare (locale o remoto).
Destinazione: dove copiare il file (locale o remoto).

Formato destinazione remota

```
utente@host:/percorso/
```

Comando scp - esempi

Copiare un file dal locale al server remoto

```bash
scp documento.txt user@192.168.1.10:/home/mario/
```

Copia documento.txt nella cartella /home/mario/ sul server remoto

---

Copiare un file dal remoto al locale

```bash
scp user@192.168.1.10:/home/mario/documento.txt .
```

Scarica documento.txt dalla home del server nella directory corrente .

```bash
scp user@192.168.1.10:/home/mario/documento.txt /home/mario_pc_locale/cartellaX
```

Scarica documento.txt dalla home del server nella directory indicata con percorso assoluto /home/mario_pc_locale/cartellaX

Redirect stdout

bash

```bash
echo "Ciao mondo" > output.txt
```

bash

```bash
echo "Altra riga" >> output.txt
```

Redirect stdin

bash

```bash
wc -l < output.txt
```

bash

```bash
cat << EOF
Prima riga
Seconda riga
EOF
```

bash

```bash
wc -w <<< "uno due tre quattro"
```

Redirect

Stderr

```bash
ls /cartella_che_non_esiste 2> ops.log
```

Stdout / stderr

```bash
ls -l > mondo.log 2>&1
```

```bash
ls -l &> mondo.log
```

Device files

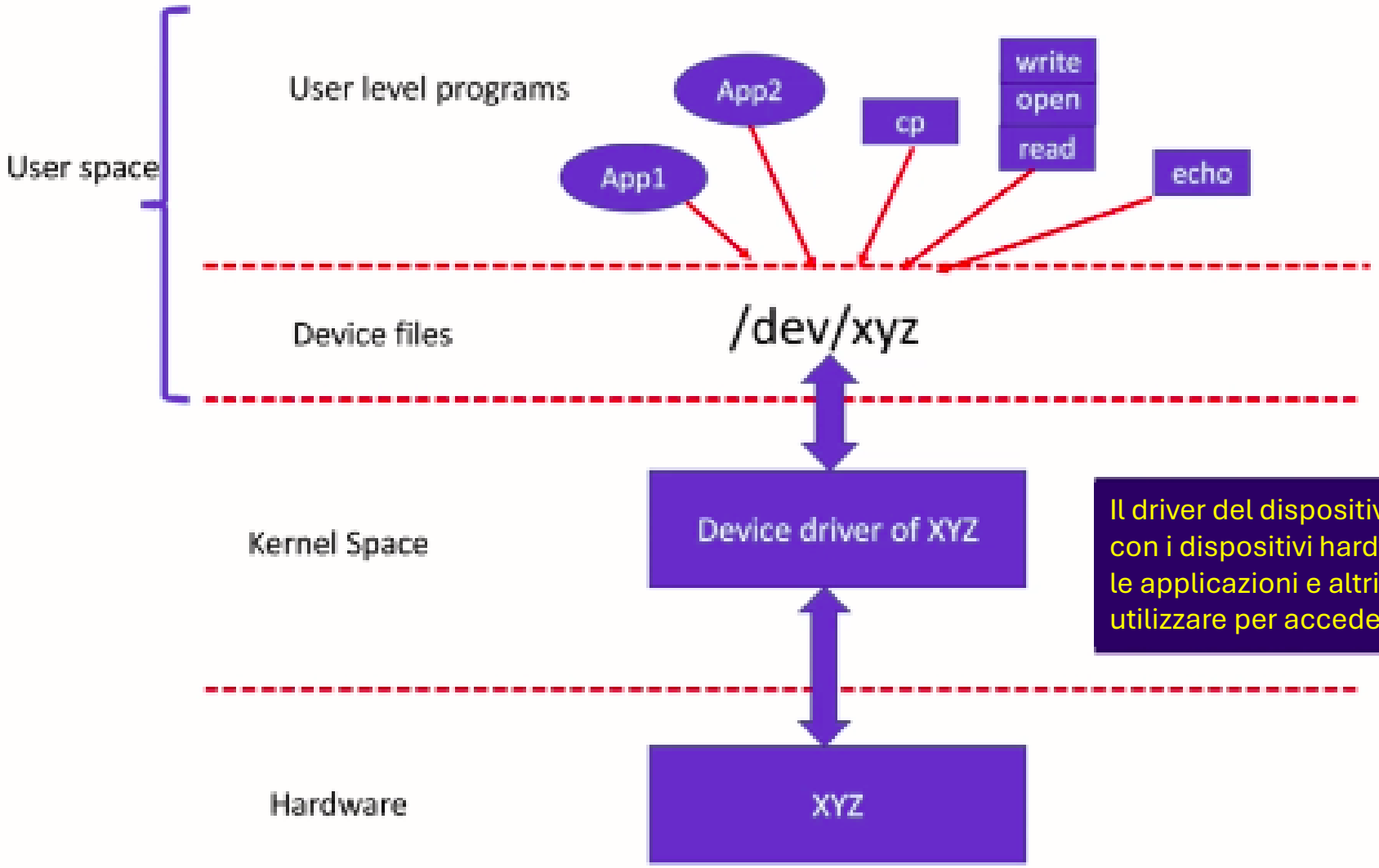
I device files sono file speciali presenti in /dev/ che rappresentano le periferiche hardware o risorse virtuali del sistema operativo.

- Permettono di interagire con i dispositivi come se fossero file normali (puoi fare read, write, open, close).
- Non contengono dati reali, ma sono un'interfaccia verso il kernel, che traduce le operazioni sui file in operazioni sul dispositivo.

Tipi principali
- Character device (c): scambio dati a flusso di byte (es. /dev/tty, /dev/zero).
- Block device (b): scambio a blocchi, tipico dei dischi (es. /dev/sda).
- Others special: pseudo-files come /dev/null, /dev/stdin, che non sono hardware ma funzionalità kernel.

Un device file è un "ponte" tra i processi e l'hardware (o risorsa virtuale), trattato come un file nel filesystem.

```
/dev/urandom
/dev/zero
/dev/null
/dev/stdin
/dev/stdout
/dev/pts/N (linux)
/dev/ttyNNN (macOS)
```



Il driver del dispositivo si fa carico dell'interazione con i dispositivi hardware ed esporta interfacce che le applicazioni e altri moduli del kernel possono utilizzare per accedere ai dispositivi.

Device files - esempi

```
/dev/urandom
/dev/zero
/dev/null
/dev/stdin
/dev/stdout
/dev/pts/N (linux)
/dev/ttyNNN (macOS)
```

bash

```bash
cat /dev/stdin
```

bash

```bash
dd if=/dev/zero bs=1 count=50 | xxd
```

bash

```bash
xdg-open 2> /dev/null
```

bash

```bash
echo ciao > /dev/pts/2
```

bash

```bash
dd if=/dev/urandom bs=1 count=50
```

bash

```bash
cat /dev/pts/4
```

Comando **du**

du sta per disk usage e serve a misurare quanto spazio su disco è occupato da file e directory.

```bash
du [OPZIONI] [FILE o DIRECTORY]
```

Opzione	**Significato / comportamento**
-h o --human-readable	Mostra le dimensioni in modo leggibile (es. K, M, G) invece che in blocchi grezzi.
-s	Summary: mostra solo il totale per ciascuna directory specificata, non il dettaglio per ogni sottodirectory.
-a	Include anche i file (non solo directory) nell'output.
-k	Mostra le dimensioni in kilobyte (o blocchi da 1 KiB) anziché nei blocchi predefiniti.
--block-size=SIZE	Forza l'output usando blocchi di dimensione "SIZE" (può essere utile per vedere tutto in MB o GB).

Comando **df**

Il comando df in Unix/Linux serve a mostrare lo spazio su disco disponibile e utilizzato sui filesystem montati.

bash

```bash
df [opzioni] [file ...]
```

bash

```bash
$ df -h
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   30G   18G  63% /
tmpfs           1.9G  2.0M  1.9G   1% /run
/dev/sdb1       100G   70G   30G  70% /mnt/data
```

Opzione	**Significato / comportamento**
-h	Mostra le dimensioni in modo leggibile (es. K, M, G)
-k	output in kilobyte.
-m	output in megabyte.
-T	include il tipo di filesystem (ext4, tmpfs, ecc.).
-i	mostra l’uso degli inode

bash

```bash
cat miofile.txt
```

```
Universit□ di Lecce
Piazza dell'Unit□
Cond□
```

Codifica

bash

```bash
cat miofile.txt
```

```
Universit□ di Lecce
Piazza dell'Unit□
Cond□
```

Comando **iconv**

Il comando iconv serve per convertire la codifica dei caratteri di un file (es. da ISO-8859-1 a UTF-8).
Molto utile per rendere leggibili testi con accenti o caratteri speciali.

bash

```bash
iconv -f CODIFICA_ORIGINE -t CODIFICA_DESTINAZIONE file_input > file_output
```

bash

```bash
iconv -f ISO-8859-1 -t UTF-8 testo_latin1.txt > testo_utf8.txt
```

bash

```bash
iconv -l
```

Comando **file**

Il comando file serve per determinare il tipo di un file, non basandosi sull’estensione ma sul contenuto.
Viene usato per capire se un file è testo, binario, con quale codifica, ecc.

bash

```bash
file [opzioni] nomefile
```

bash

```bash
file documento.pdf
```

```
documento.pdf: PDF document, version 1.4
```

Comando **sort**

Il comando sort le righe di testo di uno o più file o dallo stdin, in base a vari criteri: alfabetico, numerico, ordine inverso, colonne specifiche, ecc.
Per default, ordina le linee in ordine lessicografico/alfabetico (in base alla locale impostata) considerandole come sequenze di caratteri.

bash

```bash
sort [file ... ]
```

Opzione	Significato
-n	ordina numericamente (cioè “10” viene dopo “2”, non basandosi sui caratteri)
-r	ordine inverso (reverse)
-k	specifica la/il colonna/campo su cui ordinare (campo = parte della riga divisa da delimitatori)
-t	definisce un delimitatore diverso (default: spazi/tab) per dividere campi
-u	rimuove righe duplicate (unique)

Comando **rev**

Il comando rev prende come input una o più righe di testo (da file o da stdin) e inverte l’ordine dei caratteri in ogni riga, lasciando le righe in ordine. Prende file come argomento, altrimenti legge da input standard (stdin).

bash

```bash
rev [file ... ]
```

Creazione / cancellazione / modifica di utenti

Comando	Scopo principale	Opzioni chiave / note	Esempi / comportamento
useradd	creare un nuovo utente “a basso livello”	-m (crea home), -s (shell), -G (gruppi secondari), -g (gruppo primario), -c (commento)	sudo useradd -m -s /bin/bash mario crea utente con home e shell.
adduser	comando “alto livello” / wizard per creare utenti	su distro Debian/Ubuntu è uno script che semplifica useradd + vari controlli/configurazioni	sudo adduser mario chiede interattivamente informazioni extra (nome, password, ecc.)
usermod	modificare un utente esistente	-aG (aggiungi a gruppi), -g (modifica gruppo primario), -G (imposta gruppi secondari), -d (modifica home), -s (modifica shell), -e (scadenza)	sudo usermod -aG developers mario aggiunge “mario” al gruppo developers senza rimuoverlo dagli altri gruppi
userdel	eliminare un utente	opzioni come -r (rimuovi la home e file dell’utente)	sudo userdel -r mario elimina utente e sua home
passwd	impostare/cambiare la password di un utente	—	sudo passwd mario chiede nuova password

Gestione dei gruppi

Comando	Scopo principale	Opzioni chiave, note	Esempi, comportamento
groupadd	creare un nuovo gruppo	-g GID per specificare un ID numerico	sudo groupadd devteam oppure sudo groupadd -g 1500 devteam
groupmod	modificare un gruppo esistente	-n nuovo_nome (rinomina), -g nuovo_GID	sudo groupmod -n developers devteam per rinominare gruppo
groupdel	eliminare un gruppo	—	sudo groupdel devteam
gpasswd	gestione membri del gruppo / password del gruppo	-d per rimuovere utente da gruppo, -a per aggiungere (in alcuni sistemi)	sudo gpasswd -d mario developers rimuove “mario” dal gruppo “developers”
File correlati: /etc/group, /etc/gshadow	definizione dei gruppi e membri	struttura: groupname:password:GID: members	visualizzabile con cat /etc/group

Proprietà possesso file / directory (owner / group)

Comando	Scopo principale	Opzioni chiave, note	Esempi, comportamento
chown	cambiare l'**owner** (utente) e/o il **gruppo proprietario** di file/directory	sintassi: chown [opzioni] utente:gruppo file o utente: gruppo	sudo chown alice:developers file.txt cambia proprietario in alice e gruppo in developers
chgrp	cambiare solamente il **gruppo proprietario** di file/directory	sintassi: chgrp gruppo file	sudo chgrp developers file.txt
Opzione -R (ricorsiva)	applica l'operazione anche a directory e sottocartelle	per chown -R, chgrp -R	sudo chown -R alice:developers /home/alice
Note privilegi	Solo **root** può cambiare l'utente proprietario con chown su file non suoi; un utente può cambiare il gruppo (con chgrp) solo se è membro del gruppo di destinazione.		

Proprietà possesso file / directory (owner / group)

Comando / concetto	Scopo	Modificatori, modalità	Esempi, note
chmod	modificare i permessi di file / directory (che cosa possono fare utente, gruppo e “others”)	modalità simbolica (u, g, o, +, -, =), modalità numerica (ottale, es. 755, 644)	chmod 755 dir/ dà permessi rwx per proprietario, rx per gruppo e altri. chmod g+w file.txt aggiunge il permesso di scrittura al gruppo.
Permessi speciali: **SUID, SGID, sticky bit**	estendono il comportamento dei permessi tradizionali	spesso indicati con cifre ottali aggiuntive (es. 4xxx, 2xxx, 1xxx)	- **SUID**: esecuzione con i permessi del proprietario - **SGID**: per directory fa ereditare il gruppo alle nuove creazioni - **Sticky bit** su directory: solo il proprietario può cancellare i file al suo interno
umask	imposta il “mascheramento” (permessi di default) quando vengono creati file / directory	sottrae bit dal valore massimo (files 666, dirs 777)	ad esempio umask 022 → i nuovi file avranno permessi 644 e le directory 755.
ls -l, stat	visualizzare permessi e proprietà	ls -l file mostra permessi in forma -rwxr-xr--, con proprietario, gruppo	stat -c %U:%G file restituisce utente:gruppo proprietari

Esempi chmod

bash

`chmod u+x script.sh`	aggiunge esecuzione al proprietario
`chmod go-w file.txt`	toglie scrittura a gruppo e altri
`chmod a=r file.txt`	tutti hanno solo lettura
`chmod u=rw,g=r,o= file.txt`	utente rw, gruppo solo r, altri niente